

FIT5196 DATA WRANGLING

Week 4

Regular Expressions

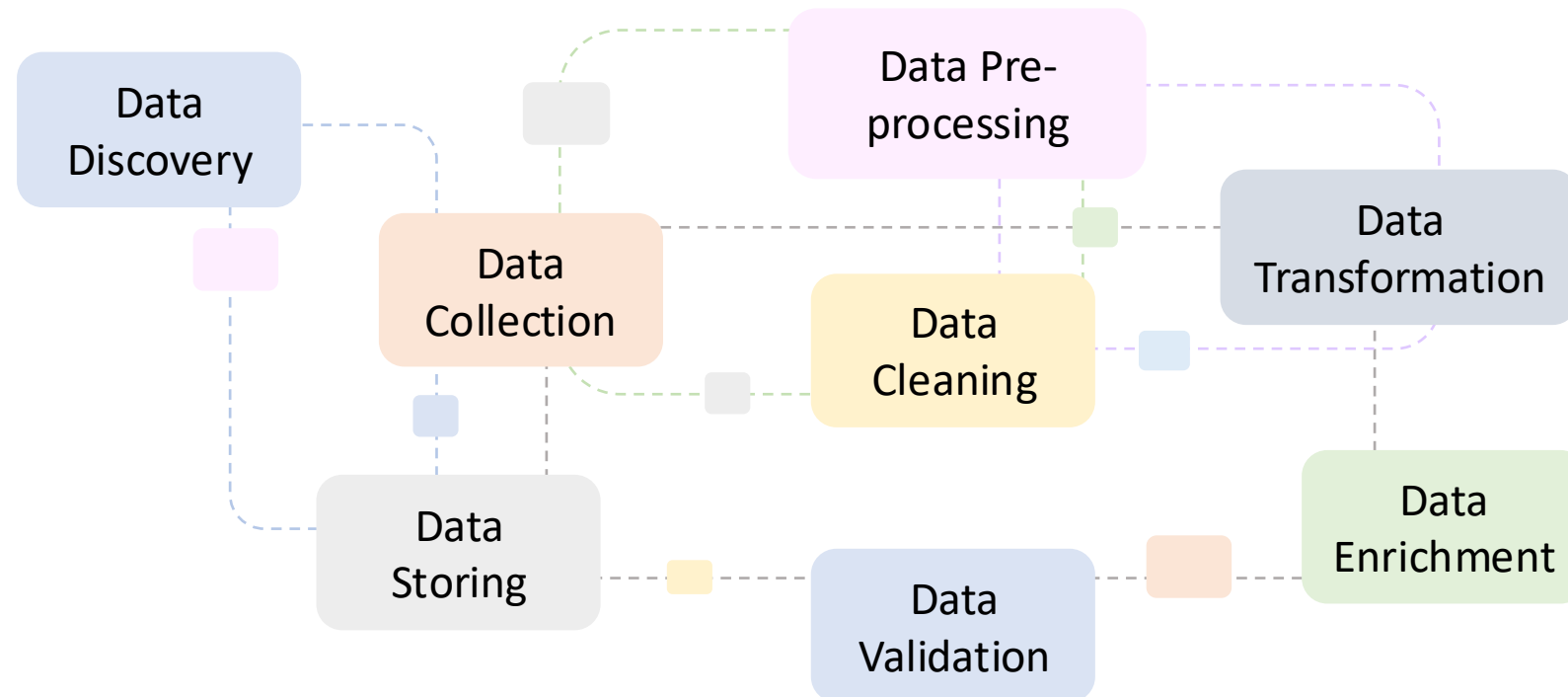
By Jackie Rong

Faculty of Information Technology

Monash University

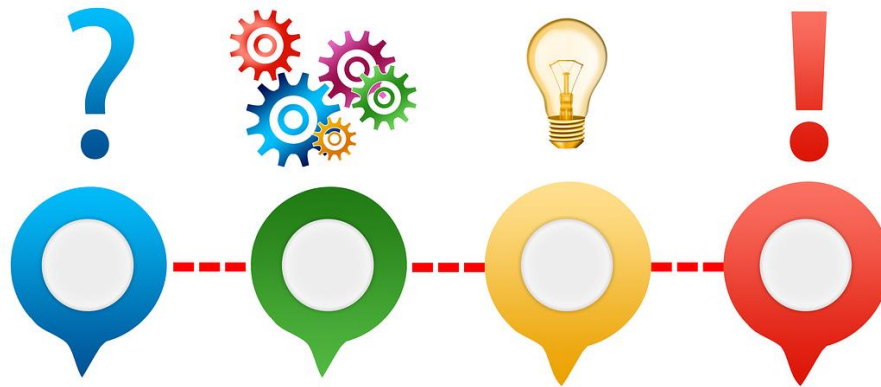
Data Wrangling Tasks

- **Data Wrangling** is the process of **acquiring**, **cleaning**, **structuring**, and **enriching** raw data into a format that is directly usable for analysis.



Outline

- What are Regular Expressions and Why?
- Regular Expression Syntax
 - Character Sets
 - Repetition
 - Grouping
 - Raw String in Python
- Text Data Pre-processing



Regular Expressions

- A **regular expression** is a set of **symbols** that describes a **text pattern**.
- Why regular expressions?
 - Regular expressions are useful in **finding**, **replacing** and **extracting** information from text, such as log files, HTML/XML files, and other documents
 - Search a document for colour or neighbour with or without 'u'
 - Convert a tab-delimited file to a comma-delimited file
 - Find duplicate words in a text
 - Search and replace "Bob" and "Bobby" with "Robert"
 - Regular expressions are useful in **verifying** whether input fits into the text pattern, such as verifying
 - phone numbers: Does a phone number have the right number of digits?
 - emails: Is an email address in a valid format?
 - date: Is a date in the right format? Does the month exceed 12?

Example – Validate Emails

`r"^[a-zA-Z0-9_+-.]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$"`¹

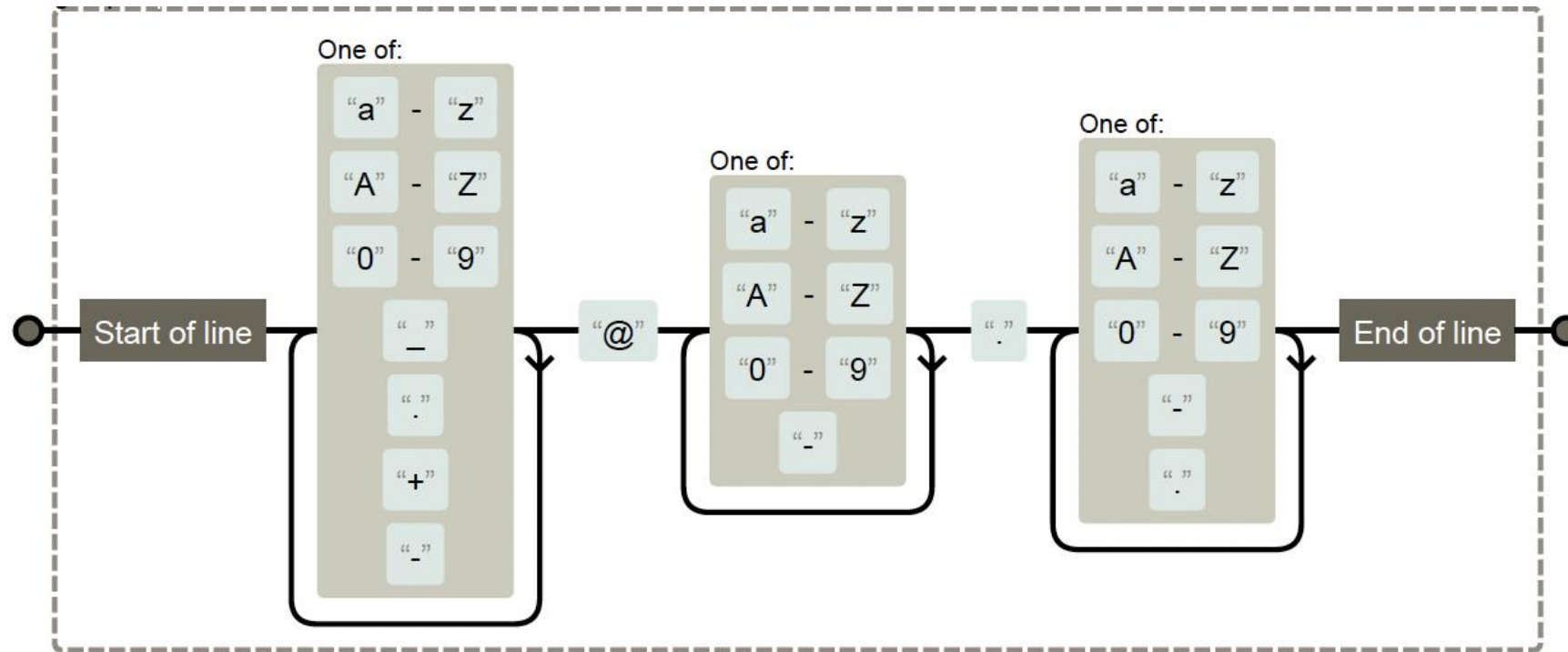
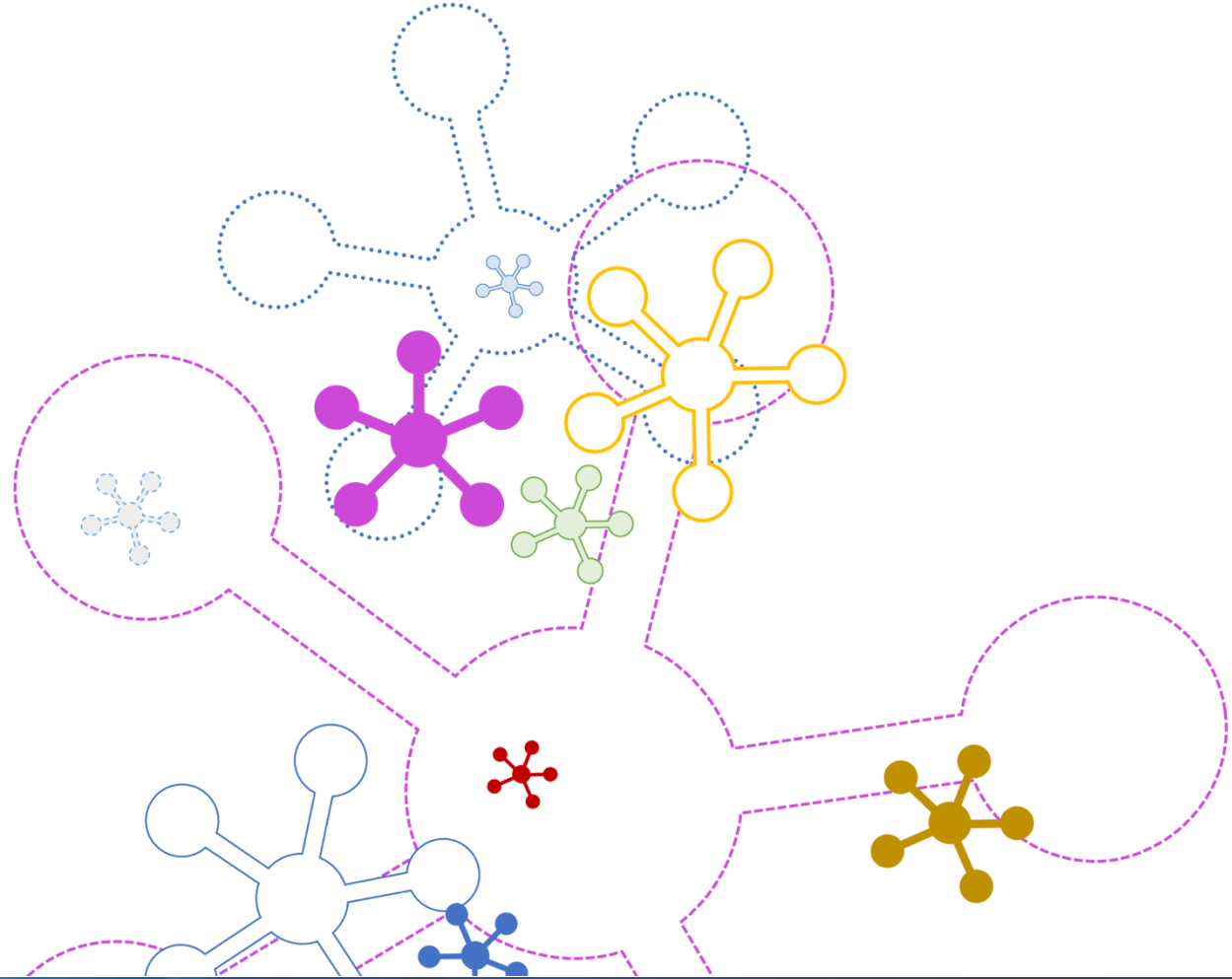


Figure: Figure generated by <https://regexper.com/>

¹<http://emailregex.com/>

Regular Expression Syntax

- Character Sets
- Repetition
- Grouping
- Raw String in Python



Character Sets

- The most obvious feature of regular expressions is **matching strings** with one or more literal characters, called string literals.
 - Everything is essentially a character in regular expressions.
 - **cat** matches “**cat**”
 - **cat** matches the first three characters of “**cattle**” and “**catfish**”
 - It is similar to searching in text editing program.
 - Matching is case-sensitive:
 - **cat** does not match “**Cat**”.
 - How does regular expression engine work?

Character Sets [...]

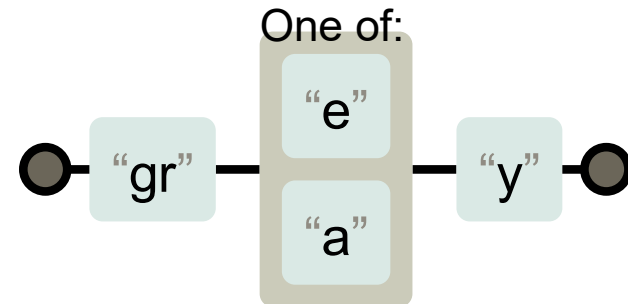
Assume that we are going to match the following two words:

grey gray

- What should the regular expression be?

[...] indicate a set of characters

- Matches any one of several characters in the set, but only one
- The order of characters does not matter.
- The regular expression is **gr[ea]y**
- **gr[ea]y** does not match **grAy**, **graay**, or **graey**.



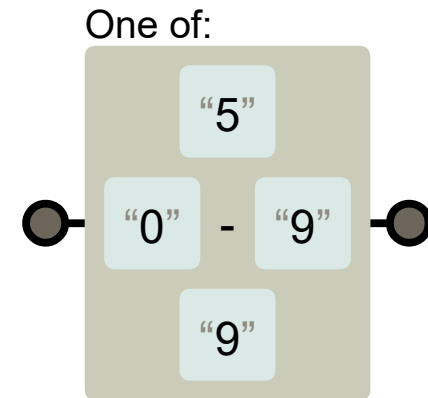
Character Ranges [a-zA-Z] and [0-9]

Assume that we are going to match Victorian car plate numbers, for example

XRA 000, 1AA 1AA

Note the letters can be from A to Z, and the numbers can be from 0 to 9. What should the regular expression be?

- Character ranges can be indicated by giving two characters and separating them by a '-'.
 - Example:
 - [0-9]
 - [a-z] or [A-Z]
- Caution
 - [50-99] is not all numbers from 50 to 99, it is the same as [0-9].



Character Ranges [a-zA-Z] and [0-9]

Assume that we are going to match victory car plate numbers, for example

XRA 000, 1AA 1AA

Note the letters can be from A to Z, and the numbers can be from 0 to 9.

What the regular expression should be?

[A-Z0-9][A-Z][A-Z]\s[0-9][A-Z0-9][A-Z0-9]



Negative Character Sets [^...]

Assume that we are going to write a regular expression that presents only the live animals from the following text

hog dog bog

what is the regular expression?

[^...]: If the first character of the set is ^, all the characters that are not in the set will be matched.

- `[^b]og` matches “hog” and “dog”, but not “bog”.
- Caution:
 - Does `see[^mn]` match “see”?
 - Does `see[^mn]` match “see ”?

Try the regular expression in Pythex (<http://pythex.org/>)!

Metacharacters Inside Character Sets `[.+]`

Assume that we are going to match the following two strings:

`var(9), var[0]`

Now, we need to match `()` and `[ ]`, how can we do that?

- Metacharacters inside character sets are already escaped. In other words, they lose their special meaning inside sets.
 - Example:
 - `h[ai.u]t` matches “hat”, “h.t”, but not “hot”
- Exceptions
 - `]`, `-`, `^` and `\` that do need to be escaped.
 - `h[ai.u]t` → `h[ai]u]t`?

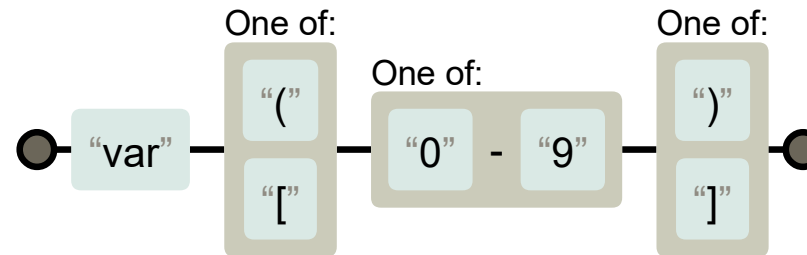
Meta-characters Inside Character Sets `[.+]`

Assume that we are going to match the following two strings:

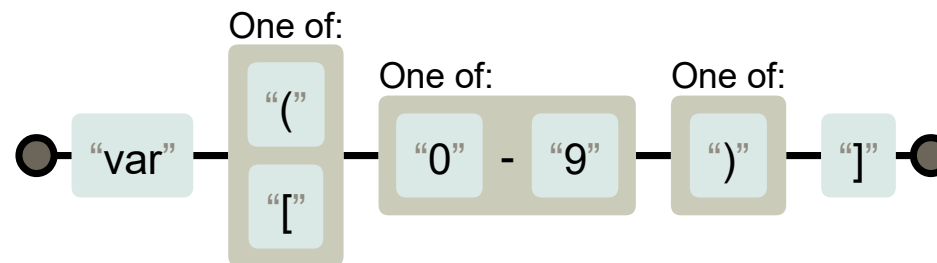
`var(9), var[0]`

Now, we need to match `()` and `[ ]`, how can we do that?

`var([([0-9](\))])`



`var([([0-9](\)])]`



Shorthand Character Sets

Shorthand	Meaning	Equivalent
<code>\d</code>	Matches any decimal digits from 0 to 9	<code>[0-9]</code>
<code>\w</code>	Matches any word character	<code>[a-zA-Z0-9_]</code>
<code>\s</code>	Matches any white space character	<code>[\t\n\r]</code>
<code>\D</code>	Matches any non-digit character	<code>[^0-9]</code>
<code>\W</code>	Matches any non-alphanumeric character	<code>[^a-zA-Z0-9_]</code>
<code>\S</code>	Matches any non-whitespace character	<code>[^ \t\n\r]</code>

- `\d\d\d\d` matches four-digit numbers, such as “2016”, but not text.
- `\w\w\w` matches three-word characters, such as “abc”, “123” and “d_b”
- `\w\w\s\w` matches “ab c” but not “a bc”.
- `[\w]-[\w]` matches two characters separated by a hyphen.
- `^\d` is the same as `[\D]`

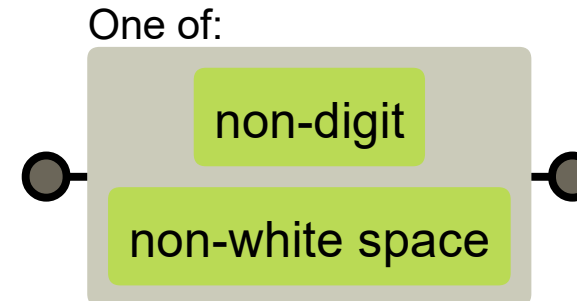
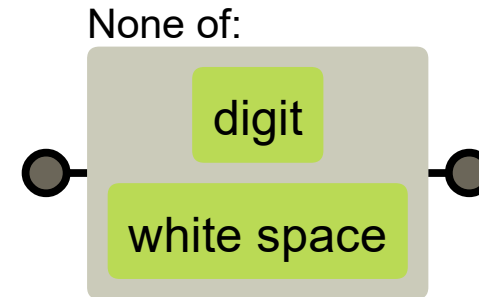
Shorthand Character Sets

- Caution:

- Is `[^\d\s]` the same as `[\D\S]`?

- `[^\d\s]`: Not digit OR space character

- `[\D\S]`: EITHER NOT digit OR NOT space character



Repetition Expression

- Repetition meta-characters

Meta-character	Meaning
*	Match 0 or more repetitions of the preceding regex
+	Match 1 or more repetitions of the preceding regex
?	Match 0 or 1 repetition of the preceding regex

Examples: Assume we are going to match the following words

oops oops oooooops oooooops

but not

ops

which regular expression(s) should we use?

- A. oo*ps
- B. ooo*ps
- C. oo+ps
- D. oo?ps

Repetition Expression

- Repetition meta-characters

Meta-character	Meaning
*	Match 0 or more repetitions of the preceding regex
+	Match 1 or more repetitions of the preceding regex
?	Match 0 or 1 repetition of the preceding regex

Examples: Assume we are going to match the following words

oops ooops oooooops ooooooops

but not

ops

which regular expression(s) should we use?

- A. oo*ps
- B. ooo*ps
- C. oo+ps
- D. oo?ps



Repetition Expression

- Repetition meta-characters

Meta-character	Meaning
*	Match 0 or more repetitions of the preceding regex
+	Match 1 or more repetitions of the preceding regex
?	Match 0 or 1 repetition of the preceding regex

Examples: Assume we are going to match the following words

oops ooops oooooops ooooooops

but not

ops

which regular expression(s) should we use?

- A. oo*ps
- B. ooo*ps
- C. oo+ps
- D. oo?ps



Repetition Expression

- Repetition meta-characters

Meta-character	Meaning
*	Match 0 or more repetitions of the preceding regex
+	Match 1 or more repetitions of the preceding regex
?	Match 0 or 1 repetition of the preceding regex

Examples: Assume we are going to match the following words

oops ooops oooooops ooooooops

but not

ops

which regular expression(s) should we use?

- A. oo*ps
- B. ooo*ps
- C. oo+ps
- D. oo?ps



Repetition Expression

- Repetition meta-characters

Meta-character	Meaning
*	Match 0 or more repetitions of the preceding regex
+	Match 1 or more repetitions of the preceding regex
?	Match 0 or 1 repetition of the preceding regex

Examples: Assume we are going to match the following words

oops oops oooooops oooooops

but not ops

which regular expression(s) should we use?

- A. oo*ps
- B. ooo*ps
- C. oo+ps
- D. oo?ps



Try the regular expression in Pythex!

Repetition Expression

- Quantified repetitions
- `{m, n}`: matches exactly from m to n repetitions of the preceding regular expression.
 - `m` (min) and `n` (max) are positive numbers
 - `m` must be always be included, can be 0
 - `n` is optional
- Three syntax
 - `\d{2}` matches numbers with exactly 2 digits.
 - `\d{2, 4}` matches numbers with 2 to 4 digits.
 - `\d{2, }` matches numbers with at least 2 digits (`n` is infinite).

Try the “oops” example in Pythex, but with `{m, n}`

Repetition Expression

Suppose we are going to match the following

report_2016_09

budget_16_08

assignment_2016_9

assignment_08_7

but not

report_201609_39

budget_2345678_08

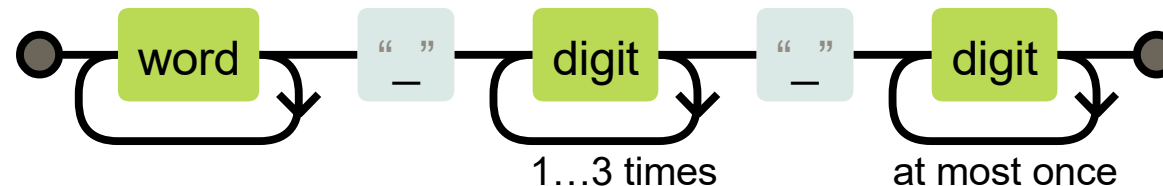
assignment_6_9000

assignment_000999_7

what is the regular expression?

- (

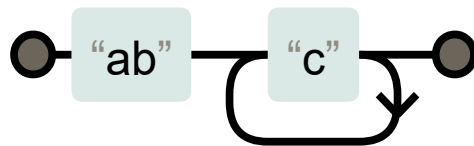
`\w+_d{2,4}_d{1,2}`



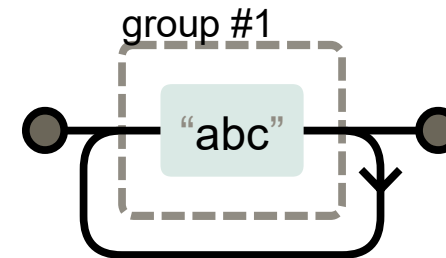
Grouping (...)

- (...) matches whatever regular expression is inside the parentheses and indicates the start and end of a group.
 - Apply repetition operators to a group of regular expressions
 - Makes regular expressions easier to read
 - Capture groups for use in matching, replacing and extraction, i.e., the contents of a group can be retrieved.
 - Cannot be used inside a character set.
- For example,

abc+ matches abc, abcc, abcccc

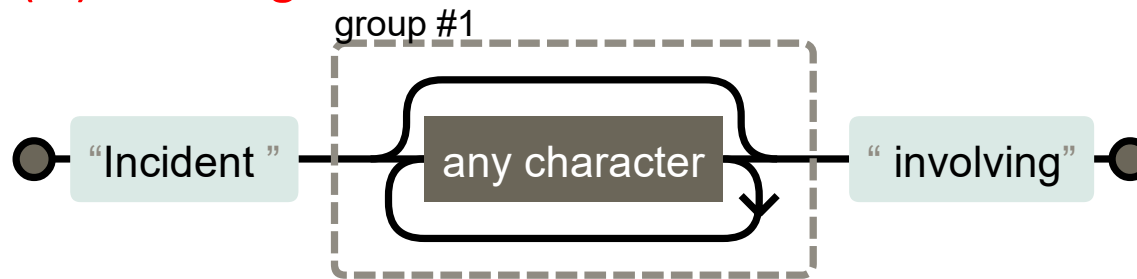


(abc)+ matches abc, abcabc, abcabcabc



Grouping (...)

- (...) matches whatever regular expression is inside the parentheses and indicates the start and end of a group.
 - Apply repetition operators to a group of regular expressions
 - Makes regular expressions easier to read
 - Capture groups for use in matching, replacing and extraction, i.e., the contents of a group can be retrieved.
 - Cannot be used inside a character set.
- For example,
 - "Incident **American Airlines Flight 11** involving a Boeing 767-223ER in 2001"
 - Regular expression: **(.*) involving**



Try it with python script from
<https://regex101.com/> !!!

Alternation |

- “|” is an OR operator
 - `A|B` will match any string that matches either A or B
 - Ordered: leftmost expression gets precedence.
 - Multiple patterns can be daisy-chained.
 - Group alternation expressions to keep them distinct.
- Examples:
 - `apple|orange` matches “apple” and “orange”
 - `(apple|orange)` juice matches “apple juice” and “orange juice”
 - `w(ei|ie)rd` matches both “weird” and “wierd”.

The Backslash Plague \

- The back slash \ indicates special forms or to allow special characters to be used without invoking their special meaning.

Character	Stage
<code>\section</code>	Text string to be matched
<code>\\section</code>	Escaped backslash for <code>re.compile()</code>
<code>\\\\section</code>	Escaped backslashes for a Python string literal

- So, to match a literal backslash, one has to write `'\\\\'` as the regular expression string
- Can we simplify the expression?

Raw String `r"..."`

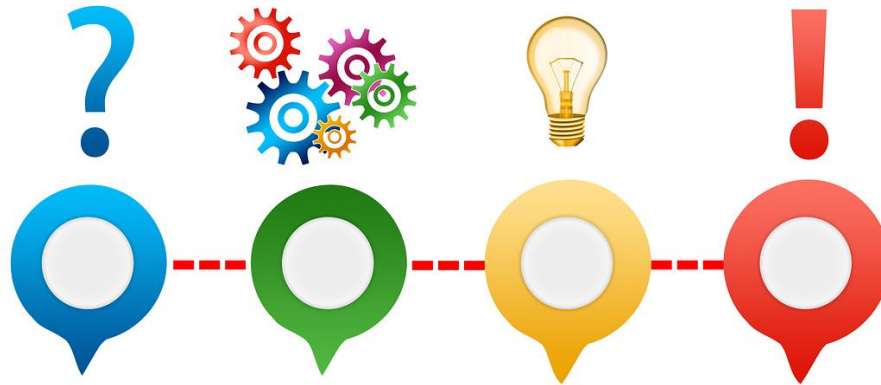
- Raw String suppress actual meaning of escape characters, and do not treat the backslash as a special character at all.

Regular Python String Literal	Raw String
<code>"\\\\section"</code>	<code>r"\\section"</code>
<code>"\\w+\\s+"</code>	<code>r"\\w+\\s+"</code>

- Regular expressions will often be written in Python code using this raw string notation.

Outline

- Exploratory Data Analysis (EDA)
 - EDA for Numerical Data
 - EDA for Text Data
- **Text Data Pre-processing**



Data Pre-processing for Text Data

- Encoding techniques

- **One-hot encoding** is a method used to convert categorical data into a binary (0 or 1) matrix.
 - Creating a binary column for each category of the variable and assigning a 1 or 0 in those columns for each data point, depending on which category is present.

- For example,

Imagine we have a dataset with a categorical feature "Colour" that includes three categories: Red, Blue, and Green.

Item	Color
1	Red
2	Blue
3	Green
4	Red

One-hot encoding



Item	Color_Red	Color_Blue	Color_Green
1	1	0	0
2	0	1	0
3	0	0	1
4	1	0	0

Data Pre-processing for Text Data (cont.)

- Encoding techniques

- **Label encoding** replaces the categorical values with a sequence of integers (e.g., 0, 1, 2, ...).
 - This method is particularly useful when the categorical variable is ordinal (i.e., the categories have a meaningful order or ranking).
 - For example,

Consider a dataset with a categorical feature "Size" that includes three categories: Small, Medium, and Large.

Item	Size
1	Small
2	Medium
3	Large
4	Small

Label encoding



Item	Size
1	0
2	1
3	2
4	0

Data Pre-processing for Text Data (cont.)

- **Vectorization methods**

- **Count vectorizer** is a method used in text processing that converts a collection of text documents into a matrix of token counts.
 - The primary idea is to represent text data numerically by counting the frequency of each word in the document, allowing algorithms to process natural language data.
 - How it works?
 1. **Tokenization:** The text is split into tokens (typically words or phrases).
 2. **Vocabulary Building:** A vocabulary of all unique tokens in the text corpus is created.
 3. **Count Calculation:** For each document, the frequency of each word in the vocabulary is counted and represented in a vector, where each dimension corresponds to a term in the vocabulary.

Example: Count Vectorizer

- Consider a simple dataset of two text documents:

1. "The cat sat on the mat."

2. "The dog sat on the log."

Step 1: Tokenise and normalise the text (convert to lowercase, remove punctuation).

- Document 1: ["the", "cat", "sat", "on", "the", "mat"]
- Document 2: ["the", "dog", "sat", "on", "the", "log"]

Step 2: Build a vocabulary of unique terms.

- Vocabulary: ["the", "cat", "sat", "on", "mat", "dog", "log"]

Step 3: Create a document-term matrix with the counts of each word appearing in each document.

	the	cat	sat	on	mat	dog	log
Doc 1	2	1	1	1	1	0	0
Doc 2	2	0	1	1	0	1	1

Tokenisation

- **Tokenization** is a fundamental step in natural language processing (NLP) and text analysis.
- A **text corpus** is broken down into smaller units, such as words, phrases, symbols, or other meaningful elements called **tokens**.
- The process helps in transforming text into a form that can be easily analysed or processed by algorithms.
- Tokenization is crucial for tasks like sentiment analysis, topic modelling, text classification, and many others, as it enables the conversion of unstructured text into a structured form.
- **Types of tokenisation**
 - Word Tokenization
 - Sentence Tokenization
 - Subword Tokenization
 - Character Tokenization

Tokenisation (cont.)

- **Importance of Tokenization**
 - **Pre-processing for NLP Tasks:** Tokenization is often the first step in preparing text for NLP tasks, allowing for further processing such as part-of-speech tagging, syntactic parsing, and entity recognition.
 - **Handling Large Texts:** By breaking down large texts into manageable units, tokenization makes it easier to analyse and process text data efficiently.
 - **Improving Model Performance:** Proper tokenization can significantly impact the performance of NLP models by ensuring that the input data is in a suitable format for analysis.
 - **Flexibility and Adaptability:** Different tokenization strategies can be adopted depending on the specific requirements of the task or the characteristics of the language being processed.

Data Pre-processing for Text Data (cont.)

- **Case Normalisation**

- Capitalisation helps readers differentiate
 - Common nouns: writer, teacher, cookies, ...
 - Proper noun: Herman Melville, Snoopy, University of Melbourne, ...
- Case normalisation converts all the words into either uppercase or lowercase words.
- *Case normalisation is not always needed!!!*
 - Information Retrieval: “Data Wrangler” v.s. "data wrangler"
 - Named Entity Recognition: One would better keep capitalised words left as capitalised.
 - A word may change its meaning
- One function to finish case normalisation: **lower()**

✓
0s

```
▶ text = "Case Normalisation for Text Data Pre-processing!"  
  text = text.lower()  
  print(text)
```

```
↳ case normalisation for text data pre-processing!
```

Data Pre-processing for Text Data (cont.)

- **Removing stop words**
 - **Stop words** are words that are extremely common and carry little lexical content.
 - They are often function words in English, for example,
 - articles (e.g., "a", "the", and "an")
 - pronouns (e.g., "he", "him", and "they")
 - particles (e.g., "well", "however" and "thus")
 - Stop words usually refer to the most common words in a language.
 - The general strategy for determining whether a word is a stop word or not is to compute its total number of appearances in a corpus.

Data Pre-processing for Text Data (cont.)

- **Removing stop words**
 - Why should we remove stop words?
 - Stop words usually appear to be of little value and have little impact on the final results, as the presence of stop words in a text document does not really help distinguishing it from other documents.
 - Failing to remove those common words could lead to skewed analysis results.
 - For example, Email analysis: remove headers (e.g., "Subject", "To", and "From"), remove a lengthy legal disclaimer

Data Pre-processing for Text Data (cont.)

- **Stemming**

- The process of identification and removal of prefixes, suffixes, and pluralisation, which leaves you with a stem.
 - 'watches -> watch'
 - 'parties -> party'
 - 'carrying -> carry'

Data Pre-processing for Text Data (cont.)

- **Lemmatization**

- A more advanced form of stemming that makes use of, for example, the context surrounding the words, an existing vocabulary, morphological analysis of words and other grammatical information (e.g., part-of-speech tags) to determine the basic or dictionary form of a word, which is known as the lemma.

```
▶ from nltk.stem import WordNetLemmatizer

# Initialize the WordNetLemmatizer
lemmatizer = WordNetLemmatizer()

# Lemmatize some words
print("Lemmatized Dog:", lemmatizer.lemmatize("dogs"))
print("Lemmatized Running (as a verb):", lemmatizer.lemmatize("running", pos='v')) # 'v' denotes verb
```

```
⇒ Lemmatized Dog: dog
   Lemmatized Running (as a verb): run
```

Summary & To-do List

- Please download and read the materials provided on Moodle.
- Review the content learnt from Week 4.
- Assessments
 - Form a group for Assessment 1
 - Continue Assessment Stage 1 and record your collaboration, if any.
- Next week: Data structure & storing